

Intelligent Adaptive Traffic Signal Control Using Computer Vision with Emergency Vehicle Priority

**A Project Report submitted to
MOHAN BABU UNIVERSITY**

**in Partial Fulfillment of the Requirements for the Award of the degree of
BACHELOR OF TECHNOLOGY**

IN

COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

Submitted by

MANNURU TEJASWANI	22102A030059
PANDETI GURU SAI RAJU	22102A030089
BALIJE PALLI MEGHANA	22102A030010
NEELAPU HERAMBUDU	22102A030080

Under the Guidance of
Mr. P. Yogendra Prasad
Assistant Professor
Department of Data Science



Department of Data Science
SCHOOL OF COMPUTING
MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati – 517 102, A.P., INDIA
2025-26



MBU MOHAN BABU UNIVERSITY

MOHAN BABU
UNIVERSITY

Sree Sainath Nagar, Tirupati – 517 102, A.P., INDIA

DEPARTMENT OF DATA SCIENCE

CERTIFICATE

This is to certify that the project report entitled

**“Intelligent Adaptive Traffic Signal Control Using
Computer Vision with Emergency Vehicle Priority”**

is the Bonafide work done by

MANNURU TEJASWANI	22102A030059
PANDETI GURU SAI RAJU	22102A030089
BALIJE PALLI MEGHANA	22102A030010
NEELAPU HERAMBUDU	22102A030080

in the Department of **Data Science**, and submitted to Mohan Babu University, Tirupati in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Data Science) during the academic year 2025-2026. This work has been carried out under my supervision. The results of this project work have not been submitted to any university for the award of any degree or diploma.

Guide:

Mr. P. Yogendra Prasad
Assistant Professor
Dept. of Data Science

Head:

Dr. K. Reddy Madhavi
Professor & Head
Dept. of Data Science

INTERNAL EXAMINER

EXTERNALEXAMINE

DEPARTMENT OF DATA SCIENCE

Vision

To become a Centre of Excellence in Data Science by imparting high quality education through teaching, training and research

Mission

- ❖ To impart quality education in Computer Science and Engineering with specializations in Data Science by disseminating knowledge through contemporary curriculum, competent faculty and effective teaching-learning methodologies.
- ❖ Nurture research, innovation and entrepreneurial skills among students and faculty to contribute to the needs of industry and society.
- ❖ Inculcate professional attitude, ethical and social responsibilities for prospective and promising Engineering profession.
- ❖ Encourage students to engage in life-long learning by creating awareness of the contemporary developments in Computer Science and Engineering with specialization in Data Science.

PROGRAM EDUCATIONAL OBJECTIVES

After few years of graduation, the graduates of B. Tech. CSE(DS) will:

- PEO1.** Pursue higher studies in Computer Science, Data science or Management.
- PEO2.** Become successful entrepreneurs or be employed by acquiring required skill sets in the domains of Data Science and allied areas.
- PEO3.** Exhibit progression and effective adaptation to technological developments through life-long learning to address ever changing industrial requirements and follow ethical attitude in professional practice.

PROGRAM SPECIFIC OUTCOMES

On successful completion of the Program, the graduates of B. Tech. CSE(DS) program will be able to:

- PSO1.** Apply appropriate data analytical techniques for building effective decision-making systems.
- PSO2.** Develop intelligent systems using novel Machine Learning and Artificial Intelligence techniques.
- PSO3.** Design and develop efficient software systems using modern tools, techniques, and platforms to meet societal needs.
- PSO4.** Apply suitable tools and techniques to build secure distributed systems.

PROGRAM OUTCOMES

On successful completion of the Program, the graduates of B.Tech. CSE (DS) Program will be able to:

- PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- PO7. Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- PO8. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- PO9. Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- PO10. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

- PO11. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- PO12. Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE OUTCOMES (CO'S)

After successful completion of this course, the students will be able to:

- CO1** Create/Design algorithms and software to solve complex Computer Science, Information Technology and allied problems using appropriate tools and techniques following relevant standards, codes, policies, regulations and latest developments.
- CO2** Consider society, health, safety, environment, sustainability, economics and project management in solving complex Computer Science, Information Technology and allied problems.
- CO3** Perform individually or in a team besides communicating effectively in written, oral and graphical forms on Computer Science, and Information Technology based systems or processes.

Mapping of Course Outcomes with COs and PSOs:

[illegible]

DECLARATION

We hereby declare that this project report titled “**Intelligent Adaptive Traffic Signal Control Using Computer Vision with Emergency Vehicle Priority**” is agenuine work carried out by us, in **B.Tech (*Computer Science and Engineering (Data Science)*)** degree course of **Mohan Babu University, Tirupati** and has not been submitted to any other course or University for the award of any degree by us.

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in our submission. We understand that any violation of the above will cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signature of the students

- 1.
- 2.
- 3.
- 4.

ACKNOWLEDGEMENT

We are extremely thankful to our beloved Chancellor and founder **Dr. M. Mohan Babu** who took keen interest to provide us the infrastructural facilities for carrying out the project work.

I am extremely thankful to our beloved Pro Chancellor **Sri Vishnu Manchu** of Mohan Babu University who took keen interest in providing better academic facilities in the institution.

We are highly indebted to **Dr. P. V. Ramana**, Vice Chancellor of Mohan Babu University for his valuable support and guidance in all academic matters.

We are very much obliged to **Dr. K. Reddy Madhavi**, Professor & Head, Department of Data Science, for providing us the guidance and encouragement in completion of this project.

We would like to express our indebtedness to the project Convener, **Mr. P. Yogendra Prasad**, Assistant Professor, Department of Data Science for his valuable guidance during the course of project work.

We would like to express our deep sense of gratitude to **Mr. P. Yogendra Prasad**, Assistant Professor, Department of Data Science, for the constant support and invaluable guidance provided for the successful completion of the project.

We are also thankful to all the faculty members of the Data Science Department, who have cooperated in carrying out our project. We would like to thank our parents and friends who have extended their help and encouragement either directly or indirectly in completion of our project work.

Project Associate

M Tejaswani
P Guru Sai Raju
B Meghana
N Herambudu

ABSTRACT

This project aims to design a Smart Traffic Light Control System that optimizes intersection signal timing using machine learning algorithms and vehicle counting techniques. Instead of relying on physical sensors, the system processes real-time video feeds to count vehicles at each lane of an intersection. By accurately determining vehicle density, the system dynamically adjusts signal phases and durations to accommodate changing traffic flows and minimize congestion. The core components include real-time video capture, vehicle detection and counting using computer vision algorithms, machine learning-based traffic pattern analysis, and adaptive signal control logic. The real-time analysis module processes video data to extract vehicle counts, which are then fed into machine learning models to predict short-term traffic demand. Based on these predictions, the adaptive signal control mechanism allocates green light durations proportionally to traffic density, prioritizing heavily congested lanes. By coordinating signal timings across adjacent intersections, the system ensures smoother traffic flow, reduces unnecessary delays, and minimizes vehicle idling time. Beyond congestion reduction, this approach improves road safety, lowers emissions, and enhances overall transportation efficiency.

Keywords : *Adaptive traffic Signal, Machine learning, Computer Vision, Artificial Intelligence (AI), Vehicle Detection, Vehicle Counting, Intersection Control.*

TABLE OF CONTENTS

DECLARATION	i
ACKNOWLEDGEMENTS	ii
ABSTRACT	iii
TABLE OF CONTENTS	iv-v
LIST OF FIGURES	vi
LIST OF TABLES	vii
CHAPTER 1: INTRODUCTION	1
1.1 INTRODUCTION OF THE PROJECT	1
1.2 BACKGROUND	2
1.3 MOTIVATION	2
1.4 NEED FOR THE PROJECT	3
1.5 OBJECTIVES	3
1.6 ORGANIZATION OF THESIS	4
CHAPTER 2: LITERATURE SURVEY	5
CHAPTER 3: SYSTEM ARCHITECTURE AND DESIGN	7
3.1 INTRODUCTION	7
3.2 VIDEO ACQUISITION	9
3.3 VEHICAL DETECTION AND COUNTING	10
3.4 TRAFFIC DENSITY ANALYSIS	10
3.5 SIGNAL CONTROL	11
3.6 EMERGENCY VEHICAL PRIORITY	11
CHAPTER 4: METHODOLOGY AND IMPLEMENTATION	13
4.1 METHODOLOGY	13
4.1.1 SYSTEM WORKFLOW OVERVIEW	13
4.1.2 YOLO-BASED VEHICLE DETECTION	14
4.1.3 LANE-WISE VEHICAL COUNTING	17
4.1.4 TRAFFIC DENSITY ESTIMATION	18

4.1.5 ADAPTIVE SIGNAL TIMING LOGIC	20
4.2 IMPLEMENTATION	21
4.2.1 VIDEO INPUT PROCESSING	21
4.2.2 MODEL INTEGRATION (YOLO)	22
4.2.3 EMERGENCY VEHICAL DETECTION SETUP	24
4.2.4 SIMULATION USING PYTHON (PYGAME)	26
4.2.5 SYSTEM SETUP	29
CHAPTER 5: RESULTS AND DISCUSSIONS	42
5.1 EXPERIMENTAL RESULT	42
5.2 OUTPUT	44
CHAPTER 6: CONCLUSION AND FUTURE WORK	46
REFERENCES	48

LIST OF FIGURES

FIG NO	LIST OF FIGURES	PAGE NO
3.1	System Architecture	8
3.6	Emergency vehicle priority	12
4.1.2	YOLO -Based vehicle detection process	16
4.1.4	Traffic Density Estimation	19
4.2.2	Model Integration	24
4.2.4	Simulation Using Python (PYGAME)	28
5.1.1	Initial Traffic Flow	42
5.1.2	Increasing Traffic Density	43
5.2.1	Traffic Signal Control System Simulation	44

LIST OF TABLES

NO	TABLES	PAGE.NO
4.1.4	Density Interpretation	19

1.INTRODUCTION

1.1 Introduction to Project

Rapid urbanization and the continuous increase in the number of vehicles have created significant challenges for modern transportation systems. Cities are experiencing heavy traffic congestion, especially at road intersections, which leads to increased travel time, fuel consumption, and environmental pollution. Managing traffic efficiently has therefore become a critical requirement in today's urban environments.

Traditional traffic signal systems mostly operate on fixed-time schedules, where signal durations are predetermined based on historical data. These systems assume that traffic patterns remain constant, which is not true in real-world scenarios. Traffic conditions vary due to factors such as time of day, weather, accidents, and public events. As a result, fixed-time systems often lead to unnecessary delays and inefficient utilization of road capacity.

To overcome these limitations, sensor-based traffic control systems were introduced. These systems use devices such as inductive loop detectors, infrared sensors, and pressure sensors to detect the presence of vehicles. While they provide some level of adaptability, they come with high installation and maintenance costs. Additionally, they often provide only limited information, such as vehicle presence, rather than detailed traffic density.

With advancements in technology, computer vision and machine learning have emerged as powerful tools for traffic management. These technologies enable real-time analysis of video data captured from surveillance cameras. By processing video feeds, it is possible to detect, classify, and count vehicles accurately, even in complex traffic conditions, without relying on physical sensors.

This project focuses on developing an Intelligent Adaptive Traffic Signal Control System using computer vision techniques. The system captures real-time video data from intersections and applies deep learning algorithms to detect and count vehicles in each lane. Based on the traffic density calculated, the system dynamically adjusts the signal timings to ensure smooth traffic flow.

An important feature of the proposed system is the integration of an emergency vehicle priority mechanism. When an ambulance or other emergency vehicle is detected, the system automatically overrides normal traffic signals and provides a green signal to the respective lane. This ensures faster response times and reduces delays in critical situations, potentially saving lives.

Overall, the proposed system aims to improve traffic efficiency, reduce congestion, and

enhance road safety. It is cost-effective, scalable, and suitable for smart city applications. By combining computer vision, machine learning, and adaptive control strategies, this system provides a modern solution to the limitations of traditional traffic management approaches.

1.2 Background

Traffic management has been a crucial aspect of urban planning for many years. With the rapid increase in population and vehicle usage, traditional transportation systems are struggling to handle the growing demand. Intersections, in particular, have become major points of congestion, where inefficient signal timing leads to long queues and delays. This has highlighted the need for more advanced and intelligent traffic control mechanisms.

Initially, traffic signal systems were designed using fixed-time control strategies, where signal durations were pre-set based on historical traffic data. Although simple to implement, these systems lack flexibility and fail to respond to real-time traffic variations. As traffic conditions became more complex, actuated systems using physical sensors were introduced to improve responsiveness. However, these systems are expensive to install, require regular maintenance, and often provide limited data about actual traffic conditions.

With the advancement of image processing techniques, researchers began exploring the use of cameras for traffic monitoring. Early vision-based systems used basic methods such as motion detection and background subtraction to identify vehicles. While these approaches reduced dependency on physical sensors, they were sensitive to environmental conditions like lighting changes, shadows, and weather, and struggled to provide accurate results in dense traffic scenarios.

Recent developments in machine learning and deep learning have significantly improved the performance of traffic monitoring systems. Advanced algorithms, such as convolutional neural networks, enable accurate vehicle detection and classification from real-time video feeds. These technologies provide detailed traffic insights, including vehicle count and density, making them highly suitable for developing adaptive traffic signal systems.

1.3 Motivation

The increasing level of traffic congestion in urban areas has become a serious concern, affecting daily life, economic productivity, and environmental sustainability. Long waiting times at intersections, fuel wastage due to idling vehicles, and rising pollution levels highlight the inefficiency of existing traffic management systems. These challenges motivated the need to develop a smarter and more responsive traffic control solution.

Another key motivation for this project is the limitations of traditional traffic signal systems.

Fixed-time and sensor-based approaches fail to adapt effectively to real-time traffic conditions and often require expensive infrastructure and maintenance. There is a clear need for a cost-effective system that can accurately monitor traffic density and dynamically adjust signal timings without relying on physical sensors.

Additionally, ensuring the smooth and timely movement of emergency vehicles such as ambulances is a critical requirement in urban traffic management. Delays in emergency response can have life-threatening consequences. This project is motivated by the need to integrate an intelligent priority mechanism that detects emergency vehicles and provides them with immediate right of way, thereby improving response time and enhancing public safety.

1.4 Need of the Project

In today's rapidly growing urban environments, traffic congestion has become a major issue due to the increasing number of vehicles on the roads. Existing traffic signal systems are not efficient enough to handle dynamic traffic conditions, especially at busy intersections. This creates long delays, increases fuel consumption, and contributes to environmental pollution. Therefore, there is a strong need for an advanced traffic management system that can respond effectively to real-time traffic situations.

Another important need for this project arises from the limitations of traditional traffic control methods. Fixed-time signals do not adapt to varying traffic densities, and sensor-based systems involve high costs, frequent maintenance, and limited data accuracy. A more reliable and cost-effective approach is required that can provide accurate traffic analysis without depending on physical sensors. Using computer vision and machine learning offers a modern solution to overcome these drawbacks.

Furthermore, the growing demand for faster emergency response systems highlights the necessity of prioritizing emergency vehicles in traffic. In many cases, ambulances and other emergency services face delays due to traffic congestion, which can lead to critical consequences. This project addresses this need by incorporating an intelligent system that detects emergency vehicles and ensures their quick passage through intersections, ultimately improving public safety and saving lives.

1.5 Objectives

The primary objective of this project is to develop an intelligent adaptive traffic signal control system that can dynamically adjust signal timings based on real-time traffic conditions. By analyzing vehicle density at each lane using computer vision techniques, the system aims to reduce congestion, minimize waiting time, and improve overall traffic flow at intersections.

Another key objective is to design a cost-effective and efficient traffic monitoring solution that eliminates the need for physical sensors. The system utilizes real-time video feeds and machine learning algorithms to detect and count vehicles accurately, providing detailed traffic data for better decision-making and scalable implementation in urban areas. The project also aims to incorporate an emergency vehicle priority mechanism to ensure quick and uninterrupted movement of ambulances and other emergency services. By automatically detecting such vehicles and providing them with immediate right of way, the system seeks to enhance road safety and reduce delays in critical situations.

1.6 Organization of Thesis

This thesis is structured into multiple chapters, each addressing a specific aspect of the proposed intelligent adaptive traffic signal control system. The first chapter introduces the project by presenting the background, motivation, need, and objectives. The second chapter provides a detailed review of existing traffic management systems, including fixed-time, sensor-based, and vision-based approaches, along with recent advancements in machine learning and deep learning techniques. The third chapter explains the system architecture, describing the different modules involved in the design.

2. LITERATURE SURVEY

[1] C. Diakaki, M. Papageorgiou, and K. Aboudolas (2002)

C. Diakaki et al. proposed a network-wide traffic-responsive signal control system using multivariable control theory to optimize signal timings across urban road networks. Their approach dynamically adjusted signal phases based on real-time traffic conditions using mathematical models and feedback control mechanisms. The system aimed to minimize congestion and improve traffic flow efficiency across multiple intersections. Their work laid the foundation for modern adaptive traffic systems by introducing coordinated signal control strategies.

Drawbacks:

1. Requires complex mathematical modeling and calibration.
2. Not suitable for real-time large-scale deployment due to computational overhead.
3. Lacks integration with modern computer vision techniques.
4. Does not handle emergency vehicle prioritization.
5. Depends heavily on accurate traffic data inputs.

[2] Buch N, Velastin S, Orwell J (2011)

Buch et al. (2011) presented a comprehensive review of computer vision-based traffic monitoring systems. They explored techniques such as background subtraction, motion detection, and object tracking for vehicle detection. The study highlighted the advantages of using cameras over traditional sensors, including wider coverage and richer data extraction. This work contributed to the adoption of vision-based systems in intelligent transportation.

Drawbacks:

1. Highly sensitive to lighting and weather conditions.
2. Struggles with occlusion in dense traffic.
3. Limited accuracy in complex real-world environments.
4. Requires high-quality camera infrastructure.
5. Does not include adaptive signal control integration.

[3] Redmon J et al. (2016)

Redmon et al. (2016) introduced YOLO (You Only Look Once), a deep learning-based object detection algorithm capable of detecting multiple objects in real time. YOLO processes the entire image in a single pass, making it highly efficient for applications like traffic monitoring. It enables accurate detection of vehicles such as cars, buses, and ambulances, even in complex environments.

Drawbacks:

1. Requires high computational resources (GPU).
2. Performance may drop in extremely dense traffic conditions.
3. Needs large annotated datasets for training.
4. Detection accuracy depends on model version and tuning.
5. Does not inherently include decision-making logic for traffic control.

[4] Li L, Lv Y, Wang F-Y (2016)

Li et al. (2016) proposed a reinforcement learning (RL)-based traffic signal control system where the model learns optimal signal timings through interaction with the environment. The system adapts dynamically to changing traffic conditions and improves performance over time by maximizing rewards such as reduced waiting time and congestion.

Drawbacks:

1. Requires extensive training and simulation data.
2. Complex reward function design is needed.
3. Scalability to multiple intersections is challenging.
4. High computational cost.
5. Limited real-world deployment and emergency handling.

[5] Redmon J, Farhadi A (2018)

Redmon and Farhadi (2018) improved the YOLO model with YOLOv3, enhancing detection accuracy and performance in real-time applications. The model uses multi-scale predictions and improved feature extraction, making it suitable for detecting vehicles in varying traffic conditions.

Drawbacks:

1. Still computationally intensive.
2. Requires GPU for real-time performance.
3. May produce false positives in cluttered environments.
4. Needs fine-tuning for specific traffic datasets.
5. Does not include traffic signal optimization logic.

3. SYSTEM ARCHITECTURE AND DESIGN

3.1 System Architecture and Design

The system architecture of the proposed intelligent adaptive traffic signal control system is designed to provide an efficient and scalable solution for real-time traffic management. It integrates computer vision, machine learning, and adaptive control techniques to monitor and regulate traffic flow at road intersections. The architecture follows a modular approach, allowing each component to perform a specific function while contributing to the overall system performance.

The primary goal of the system design is to enable real-time traffic monitoring and dynamic signal control without relying on physical sensors. Instead of traditional methods, the system utilizes video cameras installed at intersections to capture continuous traffic data. This data serves as the input for further processing, enabling accurate analysis of vehicle movement and density in each lane.

The architecture consists of multiple interconnected modules, including video acquisition, vehicle detection and counting, traffic density analysis, adaptive signal control, and emergency vehicle priority. Each module is responsible for a specific task, and they work together in a coordinated manner to ensure smooth traffic flow. The modular design also makes it easier to upgrade or extend the system in the future.

The vehicle detection and counting module plays a critical role in the system by analyzing video frames using deep learning algorithms. It identifies different types of vehicles and counts them in real time, providing essential data for traffic density estimation. This information is then passed to the traffic analysis module, which calculates lane-wise density and determines the traffic load at the intersection.

Based on the analyzed data, the adaptive signal control module dynamically adjusts the signal timings. Lanes with higher traffic density are allocated longer green signals, while less congested lanes receive shorter durations. This ensures optimal utilization of road capacity and reduces unnecessary waiting time. Additionally, the system maintains fairness among lanes to prevent any lane from being ignored.

Another important component of the system design is the emergency vehicle priority module. This module detects emergency vehicles such as ambulances and immediately overrides normal signal operations to provide a clear path. The integration of this feature enhances public safety and ensures faster response times during critical situations. Overall, the proposed system architecture offers a smart, efficient, and future-ready solution for modern traffic management.

FLOW CHART

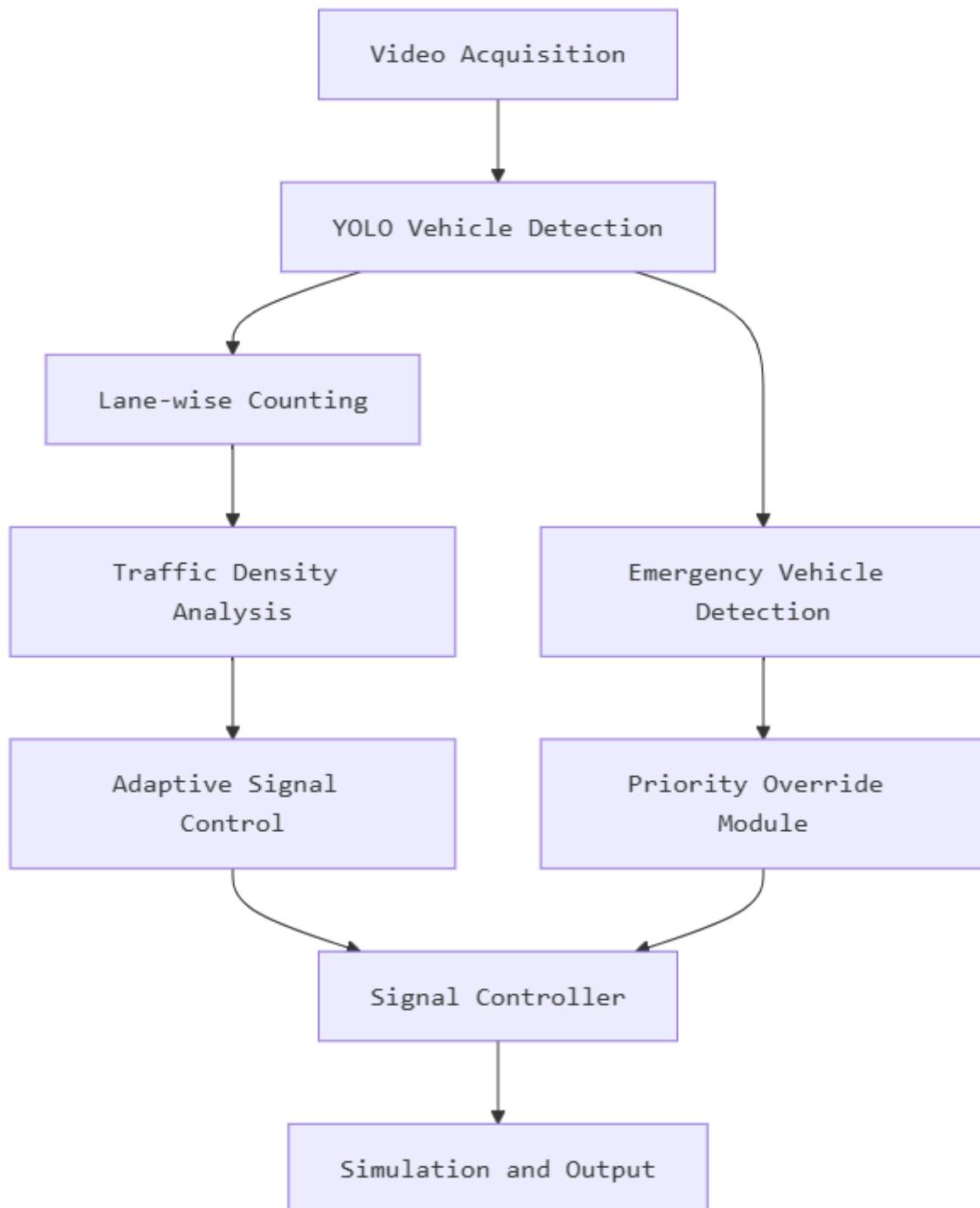


Fig 3.1 – System Architecture

3.1 Video Acquisition Module

The Video Acquisition Module is the initial stage of the intelligent traffic signal control system. It is responsible for collecting real-time traffic data using cameras installed at road intersections. This module acts as the input source for the entire system, providing continuous visual information required for traffic analysis.

1. Purpose of the Module

1. The main purpose of this module is to capture live traffic conditions and provide accurate visual data for further processing.
2. Capture real-time video of vehicles at intersections
3. Monitor all lanes continuously
4. Provide input data for vehicle detection and analysis
5. Replace traditional physical sensors

2. Camera Setup and Placement

1. Proper camera installation is essential for effective data capture.
2. Cameras are placed at strategic positions (top view or angled view)
3. Each camera covers one or more lanes of the intersection
4. Ensures full visibility of approaching and waiting vehicles
5. Avoids blind spots and overlapping issues

3. Working Process

1. The module works by continuously capturing and processing video data.
2. Cameras record live video streams from the intersection
3. Video is divided into frames (image sequences)
4. Frames are sent to the processing unit
5. Each frame is analyzed in real time by the next module

4. Key Features

1. Real-time monitoring: Continuous observation of traffic flow
2. High coverage: One camera can monitor multiple lanes
3. Non-intrusive system: No need to dig roads or install sensors
4. Scalable: Easy to add more cameras for larger intersections

5. Advantages of Video-Based Approach

1. Compared to traditional systems, this approach offers several benefits:
2. Reduces installation and maintenance costs
3. Provides detailed traffic information (vehicle count, type, movement)
4. Works efficiently in dynamic traffic conditions

5. Easily integrates with computer vision and AI models
6. Role in Overall System
 1. The Video Acquisition Module plays a foundational role in the system.
 2. Supplies input to the Vehicle Detection Module
 3. Ensures synchronized monitoring of all lanes
 4. Supports real-time decision-making for signal control
 5. Directly impacts accuracy of traffic density estimation
 6. Model Selection and Processing

3.2 Vehicle Detection and Counting

The **Vehicle Detection and Counting module** takes video frames from cameras at the junction and finds all vehicles in each frame using a YOLO deep-learning model. The frame is resized, passed through YOLO, and the model outputs bounding boxes and labels for different types of vehicles such as cars, buses, trucks, bikes, and ambulances. Low-confidence and duplicate detections are removed using a confidence threshold and Non-Maximum Suppression so that only accurate vehicle detections are kept. To count vehicles lane-wise, each lane has a defined Region of Interest (ROI) near the stop line, and the centroid of every detected bounding box is checked against these ROIs. If a vehicle's centroid falls inside a lane's ROI, the counter for that lane is increased, giving real-time lane-wise vehicle counts. These counts are used to estimate traffic density for adaptive signal control and to detect emergency vehicles like ambulances, which are sent to the priority module for special handling.

3.3 Traffic Density Analysis

The **Traffic Density Analysis module** takes the lane-wise vehicle counts from the detection and counting module and converts them into a normalized measure of congestion for each lane. It focuses on vehicles waiting near the stop line within a predefined Region of Interest (ROI), so that only the vehicles actually queuing at the signal are considered for density calculation. For each lane i , the density is computed using the formula $Density_i = V_i/V_{max}$, where V_i is the number of vehicles in that lane and V_{max} is the maximum vehicle capacity of the lane. This normalization keeps density values between 0 and 1 and allows fair comparison between lanes with different capacities. The module may also track queue length and lane occupancy over time to capture short-term traffic demand trends.

The computed density values are continuously updated and sent to the Adaptive Signal Control module, which uses them to decide how long each lane should receive a green signal. Lanes with higher density get proportionally longer green times, while low-density lanes receive

shorter green phases, improving overall efficiency. By basing signal timing on density instead of fixed schedules, the system can react quickly to changing traffic conditions such as sudden surges or drops in flow. This reduces unnecessary waiting, helps clear heavy queues faster, and supports stable traffic flow even during peak periods. In this way, the Traffic Density Analysis module forms the key link between raw vehicle counts and intelligent, data-driven signal control.

3.4 Signal Control

- The **Adaptive Traffic Signal Control module** uses lane-wise traffic density values to calculate green time for each lane.
- For each lane i , green time is computed using the formula:

$$G_i = G_{\min} + (G_{\max} - G_{\min}) \cdot \text{Density}_i$$

where $G_{\min}$ and $G_{\max}$ are the minimum and maximum green times.

- Lanes with higher density get **longer** green time, and lanes with lower density get **shorter** green time, so signal time is used efficiently.
- After calculation, the module enforces minimum and maximum limits and then assigns the green signal to the selected lane.
- A fairness-aware **round-robin** scheduling is used: lanes are served in cyclic order, but their green time length depends on their density, preventing starvation of any lane.

3.5 Emergency Vehicle Priority

The **Emergency Vehicle Priority Module** is an essential component of the intelligent traffic signal control system. Its main purpose is to ensure that emergency vehicles such as ambulances, fire trucks, and police vehicles can pass through intersections without delay. In critical situations, even a small delay can result in serious consequences, so providing immediate priority is very important. This module works by continuously monitoring the output from the vehicle detection system. Using deep learning algorithms (like YOLO), the system can identify emergency vehicles as a separate class. Once an emergency vehicle is detected within a predefined distance from the intersection, the system immediately triggers an emergency mode. In emergency mode, the normal traffic signal operation is temporarily overridden. All other lanes are turned to red, and a green signal is given only to the lane where the emergency vehicle is present. This allows the emergency vehicle to pass through the intersection quickly and safely. After the vehicle clears the intersection, the system automatically returns to normal adaptive signal operation without disturbing overall traffic flow. It eliminates the need for manual intervention or additional hardware like GPS-based systems, making it cost-effective and efficient.

Emergency Vehicle Priority Module

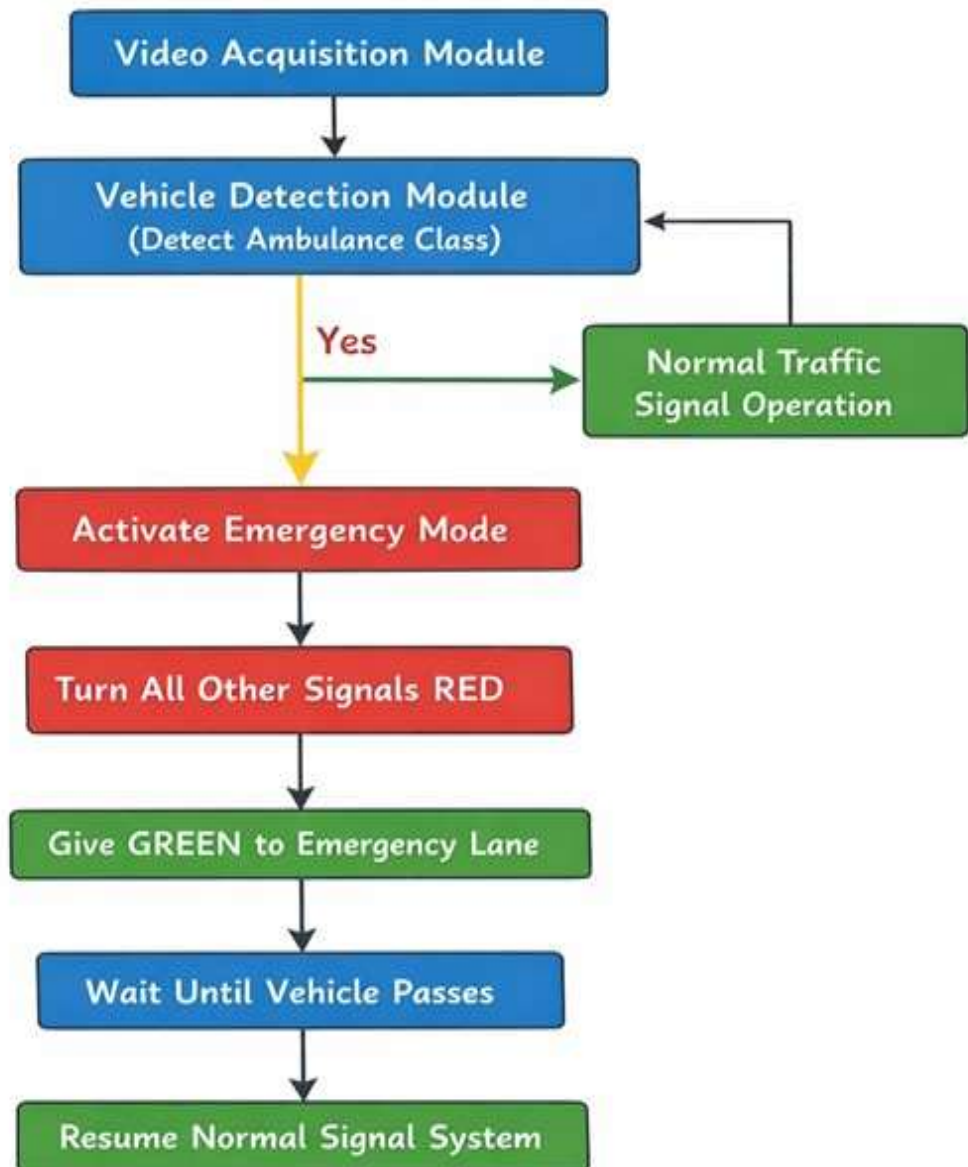


Fig 3.6 Emergency vehicle priority

4.METHODOLOGY AND IMPLEMENTATION

4.1 Methodology

The proposed system uses a computer vision-based approach to manage traffic efficiently. It begins with capturing real-time video from cameras installed at the intersection. The video is processed frame by frame, and deep learning algorithms such as YOLO are used to detect and classify vehicles. Each vehicle is counted lane-wise using predefined regions, and this data is used to calculate traffic density for every lane. Based on the computed traffic density, the system dynamically adjusts signal timings by allocating longer green signals to congested lanes and shorter durations to less crowded ones. Additionally, the system includes an emergency vehicle priority mechanism, which detects ambulances and immediately provides a green signal to their lane. This ensures smooth traffic flow, reduces waiting time, and improves overall efficiency of the traffic management system.

4.1.1 System Workflow Overview

1. Video Capture

- Cameras installed at the intersection capture real-time traffic video.

2. Frame Extraction

- The video stream is divided into frames for processing.

3. Vehicle Detection

- Each frame is processed using a deep learning model (YOLO) to detect vehicles.

4. Vehicle Classification

- Vehicles are classified into types such as cars, buses, trucks, bikes, and ambulances.

5. Lane Identification

- Predefined Regions of Interest (ROI) are used to assign detected vehicles to specific lanes.

6. Vehicle Counting

- The number of vehicles in each lane is counted in real time.

7. Traffic Density Calculation

- Lane-wise vehicle count is used to calculate traffic density.

8. Adaptive Signal Timing

- Green signal duration is dynamically assigned based on traffic density.

9. Emergency Vehicle Detection

- System continuously checks for emergency vehicles like ambulances.

10. Priority Control Activation

- If an emergency vehicle is detected, normal signals are overridden.

11. Green Signal for Emergency Lane

- Immediate green signal is given to the lane with the emergency vehicle.

12. System Reset

- After the emergency vehicle passes, the system returns to normal operation.

4.1.2 YOLO-Based Vehicle Detection

YOLO (You Only Look Once) is a powerful deep learning algorithm used for real-time object detection. In this project, it is used to detect and classify vehicles from video frames captured at traffic intersections. Its ability to process images in a single pass makes it fast and efficient, making it suitable for real-time traffic monitoring systems. The YOLO algorithm divides an input image into grids and predicts bounding boxes along with class probabilities for each grid cell. Each detected object, such as a car, bus, truck, bike, or ambulance, is identified with a confidence score. The video frames are resized and passed through the YOLO model, which outputs bounding boxes and labels for detected vehicles. To improve accuracy, techniques like confidence thresholding and Non-Maximum Suppression (NMS) are applied to remove low-confidence and overlapping detections. YOLO offers high speed and accuracy even in complex traffic conditions such as congestion, varying lighting, and occlusions. It can detect multiple vehicles in a single frame efficiently. The detected vehicle data is further used for lane-wise counting, traffic density estimation, and adaptive signal control, making it a crucial component of the intelligent traffic management system.

Algorithm: YOLO-Based Vehicle Detection

Step 1: Input Frame Acquisition

- Capture a video frame from the traffic camera at the intersection.

Step 2: Frame Preprocessing

- Resize the frame to match YOLO input size (e.g., 416×416).
- Normalize pixel values for better model performance.

Step 3: Pass Frame to YOLO Model

- Input the preprocessed frame into the trained YOLO neural network.

Step 4: Grid Division

- The image is divided into multiple grid cells.

- Each grid cell predicts possible objects.

Step 5: Bounding Box Prediction

- Each grid predicts bounding boxes around detected objects.
- It also predicts object confidence scores and class probabilities.

Step 6: Confidence Thresholding

- Remove detections with low confidence scores.
- Keep only high-probability detections.

Step 7: Non-Maximum Suppression (NMS)

- Remove overlapping bounding boxes.
- Retain only the most accurate bounding box for each object.

Step 8: Vehicle Classification

- Classify detected objects into categories such as:
 - Car
 - Bus
 - Truck
 - Bike
 - Ambulance

Step 9: Output Detection Results

- Display final bounding boxes with labels on the frame.
- Send detection results to the next module (vehicle counting).

Final Output:

- Accurate detection and classification of vehicles in real time.

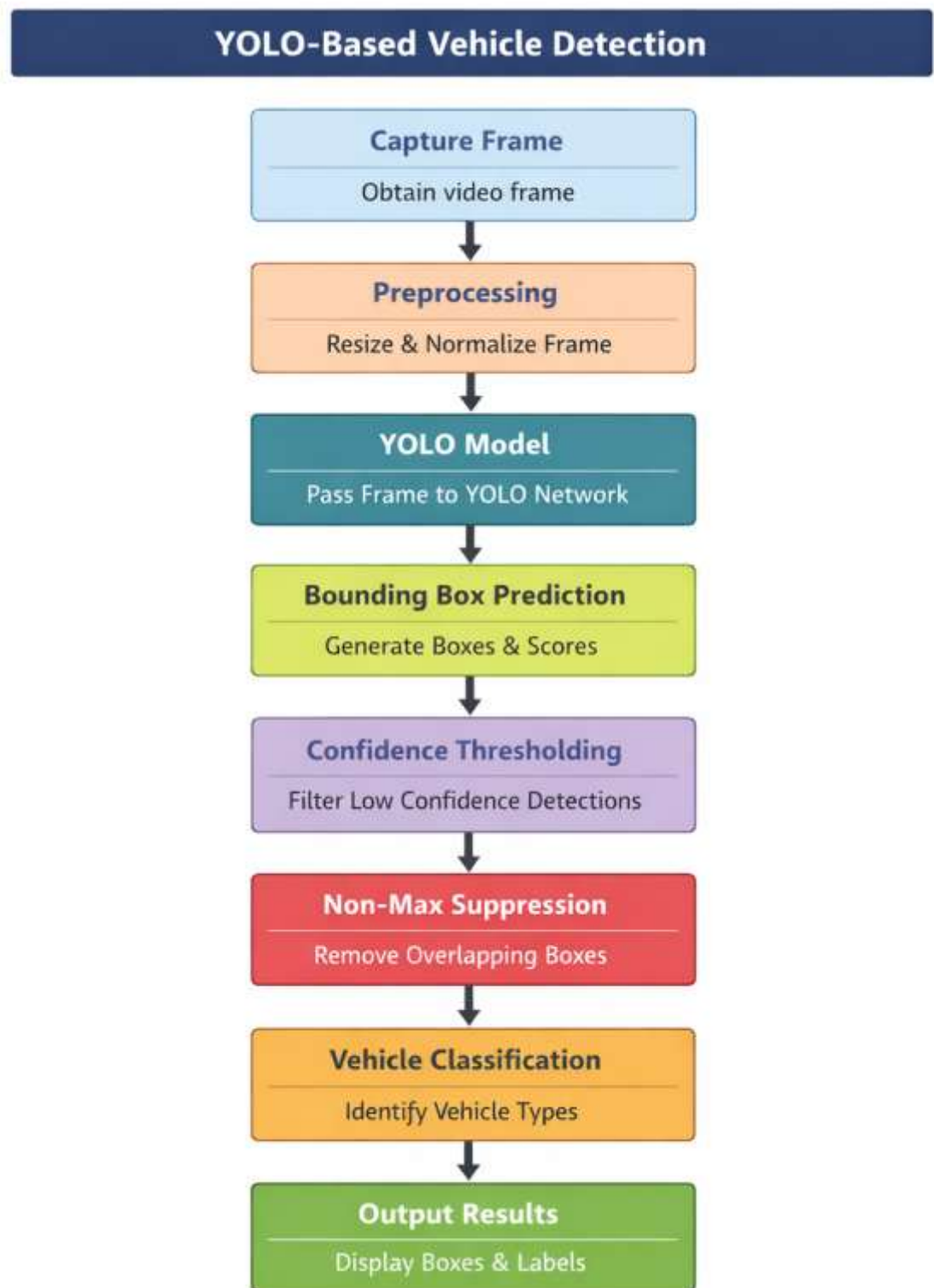


Fig 4.1.2 – YOLO -Based vehicle detection process

4.1.3. Lane Wise Vehicle Counting

Lane-wise vehicle counting determines the number of vehicles present in each lane of an intersection. After vehicles are detected using YOLO, each vehicle is assigned to a specific lane to measure traffic density accurately.

Each lane is defined using a **Region of Interest (ROI)**. The system calculates the centroid of each vehicle's bounding box and checks which ROI it falls into. The lane counter for that ROI is then incremented. This ensures precise lane-wise counting, even in multi-lane intersections. The vehicle counts are updated continuously in real time. This data is used by the traffic density analysis module to calculate congestion levels and dynamically adjust signal timings. Lane-wise counting is therefore a crucial step for adaptive and efficient traffic management.

Input:

- Video frames from intersection camera
- Detected vehicle bounding boxes from YOLO model
- Predefined Regions of Interest (ROI) for each lane

Output:

- Lane-wise vehicle count

Procedure

1. **Frame Processing**

- Receive a video frame from the video acquisition module.
- Pass the frame through the YOLO model to obtain detected vehicles.

2. **Bounding Box Handling**

- For each detected vehicle, extract bounding box coordinates.

3. **Centroid Computation**

- Compute the centroid of each bounding box:

$$(x_c, y_c) = \left(\frac{x_{min} + x_{max}}{2}, \frac{y_{min} + y_{max}}{2} \right)$$

4. **Lane Identification using ROI**

- Compare centroid position with predefined lane ROIs.
- Assign the vehicle to the lane whose ROI contains the centroid.

5. **Lane-wise Counting**

- Increment the counter of the corresponding lane.
- Repeat for all detected vehicles in the frame.

6. **Continuous Update**

- Perform the above steps for each incoming frame.
- Maintain updated vehicle counts for all lanes in real time.

7. Output Forwarding

- Send lane-wise vehicle counts to the traffic density analysis module.

4.1.4 Traffic Density Estimation

Traffic density estimation is a crucial stage in the proposed system, as it determines the level of congestion in each lane based on the number of detected vehicles. After performing lane-wise vehicle counting, the system uses the obtained counts to evaluate how crowded each lane is. This information is essential for making intelligent decisions in adaptive signal control.

In the proposed approach, traffic density is calculated by comparing the number of vehicles in a lane with a predefined threshold or maximum lane capacity. Each lane is assigned a density value, which can be represented as low, medium, or high, or as a normalized ratio. This normalization ensures fair comparison across all lanes, regardless of differences in lane length or width. The system continuously updates traffic density in real time by processing each video frame. As vehicle counts change, the density values are recalculated dynamically. This enables the system to respond instantly to traffic fluctuations such as sudden congestion or clearing of vehicles. The updated density values are then passed to the adaptive signal control module, which allocates signal timings accordingly.

This method ensures efficient utilization of road space by prioritizing lanes with higher traffic density. It reduces waiting time, minimizes congestion, and improves overall traffic flow. By integrating real-time density estimation with intelligent signal control, the system achieves a responsive and scalable solution for modern traffic management.

Given:

- Lane vehicle counts $\rightarrow C = \{c_1, c_2, \dots, c_k\}$
- Optional lane capacities $\rightarrow M = \{m_1, m_2, \dots, m_k\}$

Processing Logic

For each lane i :

$$d_i = \begin{cases} \frac{c_i}{m_i}, & \text{if capacity is defined} \\ \frac{c_i}{\max(C)}, & \text{otherwise} \end{cases}$$

Normalization

$$d_i \in [0,1]$$

- Ensures uniform comparison across all lanes

- Removes bias due to lane size differences

Density Interpretation

Density Value d_i	Traffic Level
$d_i < 0.3$	Low
$0.3 \leq d_i < 0.7$	Medium
$d_i \geq 0.7$	High

System Behaviour

- Recomputed for **every frame**
- Reflects **real-time congestion changes**
- Directly feeds into **adaptive signal timing**

Output:

$$D = \{d_1, d_2, \dots, d_k\}$$

→ Lane-wise normalized traffic density values

System Integration

$D \rightarrow$ Adaptive Signal Control Module

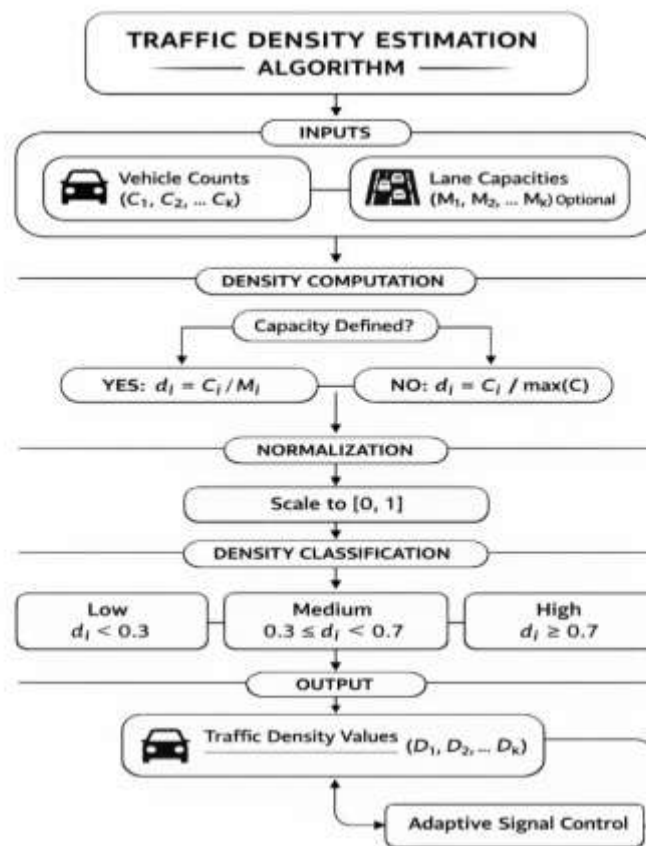


Fig 4.1.4 Traffic Density Estimation

4.1.5 Adaptive Signal Timing Logic

Adaptive signal timing is a key component of the proposed smart traffic management system, as it dynamically adjusts the duration of traffic signals based on real-time traffic density. Unlike traditional fixed-time signals, the adaptive approach ensures that lanes with higher vehicle density receive longer green signal durations, thereby reducing congestion and improving traffic flow efficiency.

In the proposed system, the traffic density values calculated for each lane are used as the primary input for signal timing decisions. The system compares density levels across all lanes and assigns priority to the lane with the highest congestion. Based on predefined minimum and maximum time limits, the green signal duration is computed proportionally to the traffic density, ensuring fair and efficient distribution of signal time. The adaptive signal timing logic operates continuously in real time. As new vehicle counts and density values are updated from the video processing module, the signal timings are recalculated dynamically. This enables the system to respond quickly to sudden traffic changes, emergency situations, or varying traffic patterns throughout the day, ultimately improving road utilization and reducing waiting time at intersections.

Algorithm: Adaptive Signal Timing Logic (Proposed System)

Input:

- Traffic density values for each lane $D = \{d_1, d_2, d_3, \dots, d_k\}$
- Minimum green signal time T_{min}
- Maximum green signal time T_{max}

Output:

- Optimized green signal duration for each lane

Logic Flow

Step 1: Receive updated traffic density values from the density estimation module.

Step 2: Identify the lane with the highest traffic density.

Step 3: Calculate signal time for each lane using proportional allocation:

$$T_i = T_{min} + \left(\frac{d_i}{\sum d} \right) \times (T_{max} - T_{min})$$

Step 4: Ensure calculated time does not exceed maximum signal limit.

Step 5: Assign green signal to the selected lane for the computed duration.

Step 6: After completion, switch to the next lane based on updated density values.

Step 7: Repeat the process continuously for real-time traffic management.

Key Features

- Dynamic signal adjustment based on traffic density
- Real-time decision making
- Reduced waiting time and congestion
- Efficient utilization of road capacity
- Scalable for multiple lanes

4.2 Implementation

The implementation phase focuses on developing and integrating the proposed smart traffic management system using computer vision and machine learning techniques. The system is designed to process real-time video input, detect vehicles, estimate traffic density, and dynamically control traffic signals. Each module is implemented sequentially and tested to ensure accurate and reliable system performance.

The system operates in a continuous loop where video frames are captured, processed, and analyzed to generate traffic-related information. This information is then used to make intelligent decisions regarding signal timing and emergency vehicle priority. The implementation ensures real-time responsiveness, scalability, and efficient traffic flow management.

4.2.1 Video Input Processing

Video input processing is the initial stage of the proposed smart traffic management system, where real-time traffic video is captured and prepared for further analysis. This stage ensures that the video data is properly formatted and optimized before being passed to the vehicle detection module. Efficient video processing is essential for maintaining system accuracy and real-time performance. In the proposed system, traffic cameras installed at road intersections continuously capture video streams. These video streams are converted into individual frames at a specific frame rate. Each frame undergoes preprocessing steps such as resizing, noise reduction, and color conversion to enhance image quality and improve detection accuracy.

Video input processing operates continuously to support real-time monitoring of traffic conditions. By ensuring consistent frame quality and proper data handling, this module plays a critical role in enabling reliable vehicle detection, lane-wise counting, and adaptive traffic signal control within the integrated system.

Algorithm: Video Input Processing

Objective

To capture real-time traffic video, convert it into frames, preprocess the frames, and prepare them

for vehicle detection

Input

- Live video stream from traffic camera

Output

- Preprocessed video frames ready for vehicle detection

Algorithm Steps

Step 1: Initialize the traffic camera or video source.

Step 2: Start capturing the live video stream.

Step 3: Read the video stream frame by frame.

Step 4: Resize each frame to the required resolution for processing.

Step 5: Convert the frame into a suitable format (e.g., RGB or grayscale).

Step 6: Apply noise reduction or image enhancement techniques if necessary.

Step 7: Store or buffer the processed frame temporarily.

Step 8: Send the processed frame to the vehicle detection module.

Step 9: Repeat the process continuously for real-time monitoring.

Step 10: Stop the process when the system is terminated.

4.2.2 Model Integration(YOLO)

Model integration refers to incorporating the trained YOLO (You Only Look Once) object detection model into the proposed smart traffic management system to perform real-time vehicle detection. The integration ensures that the model can process incoming video frames efficiently and generate accurate detection results. This step connects the deep learning model with the overall traffic monitoring pipeline.

In the proposed system, the YOLO model is loaded into the processing environment using the required libraries and configuration files. Once the model is initialized, each preprocessed video frame obtained from the video input processing module is passed to the YOLO model. The model analyzes the frame and detects vehicles by drawing bounding boxes around them and assigning class labels such as car, bus, truck, bike, or ambulance.

The output generated by the YOLO model is then forwarded to subsequent modules such as lane-wise vehicle counting and traffic density estimation. By integrating the YOLO model into the system, the proposed solution achieves fast detection speed, high accuracy, and reliable performance in real-time traffic monitoring environments.

Algorithm: Model Integration (YOLO)

Module: YOLO-Based Vehicle Detection Integration

Purpose: Integrate the trained YOLO model into the system to detect vehicles from incoming

video frames in real time.

Algorithm Format – Process-Oriented Representation

BEGIN

1. System Initialization
 - Import required libraries (OpenCV, NumPy, Deep Learning framework).
 - Set system parameters and initialize processing environment.
2. Model Loading Phase
 - Load YOLO configuration file.
 - Load trained YOLO weight file.
 - Initialize neural network model in memory.
3. Parameter Configuration
 - Define confidence threshold value.
 - Define Non-Maximum Suppression (NMS) threshold.
 - Set input frame size for detection.
4. Frame Acquisition
 - Receive preprocessed video frame from Video Input Processing module.
5. Detection Execution
 - Pass input frame to YOLO model.
 - Perform forward propagation through the neural network.
 - Generate bounding boxes and class probabilities.
6. Result Optimization
 - Filter detections based on confidence score.
 - Apply Non-Maximum Suppression to remove overlapping boxes.
7. Output Generation
 - Assign class labels to detected vehicles.
 - Store detection coordinates and vehicle information.
8. Data Transfer
 - Send detection results to Lane-wise Vehicle Counting module.
9. Continuous Operation
 - Repeat Steps 4–8 for each incoming video frame.

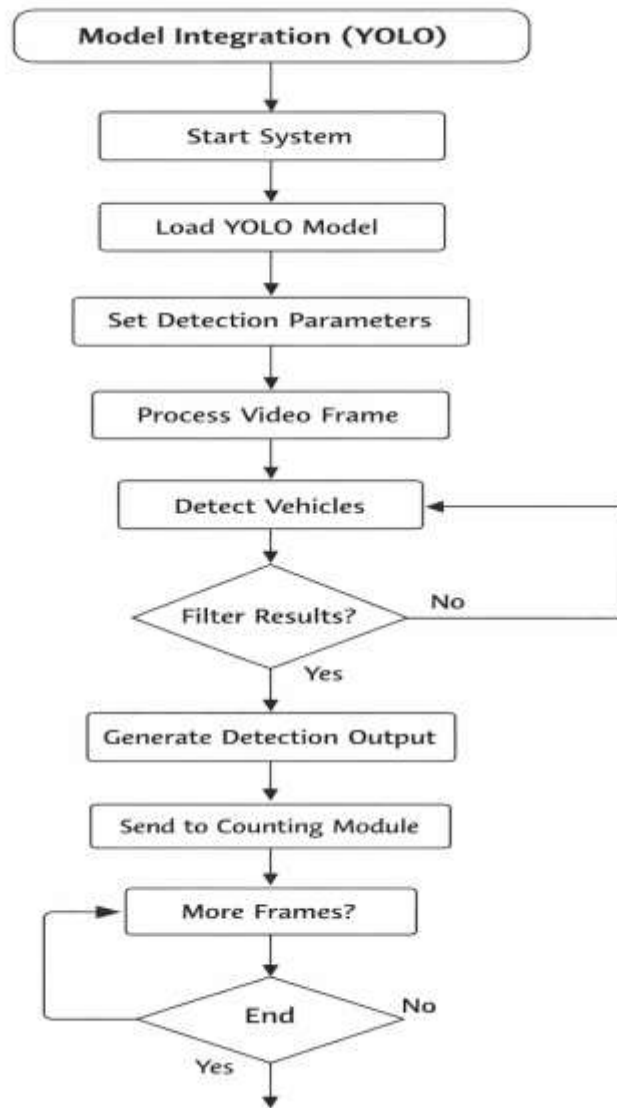


Fig 4.2.2 Model Integration

4.2.3 Emergency Vehicle Detection Setup

Emergency vehicle detection is an essential component of the proposed smart traffic management system, designed to identify emergency vehicles such as ambulances, fire trucks, and police vehicles in real time. The system ensures that these vehicles receive immediate priority at traffic intersections, reducing delays and enabling faster response during critical situations.

In the proposed system, emergency vehicles are detected using the YOLO-based object detection model, which classifies vehicles based on their visual features. When an emergency vehicle is identified in a video frame, the system generates a priority signal and temporarily overrides the normal traffic signal timing. This mechanism allows the emergency vehicle to pass through the intersection without obstruction.

The emergency vehicle detection module operates continuously alongside other system components such as vehicle detection, lane-wise counting, and traffic density estimation. By integrating emergency detection into the traffic control logic, the system improves road safety, reduces response time, and enhances the efficiency of emergency services.

System Configuration Layout

Detection Source:

- Real-time traffic video feed
- Camera positioned at road intersection

Detection Method:

- YOLO-based object detection
- Vehicle classification using trained model

Recognition Targets:

- Ambulance
- Fire truck
- Police vehicle

Decision Trigger:

- Emergency vehicle detected in lane

System Response:

- Activate priority signal
- Extend green signal duration
- Notify traffic control module

Operational Sequence (Execution-Oriented Format)

Phase 1 — Monitoring

The system continuously monitors incoming video frames from the traffic camera to observe vehicle movement at the intersection.

Phase 2 — Detection

Each frame is processed using the trained detection model to identify vehicle types and classify emergency vehicles based on learned features.

Phase 3 — Verification

Detected vehicles are validated using confidence thresholds to ensure accurate identification and reduce false detections.

Phase 4 — Priority Activation

Once an emergency vehicle is confirmed, the system automatically overrides the normal traffic signal sequence and assigns a green signal to the corresponding lane.

Phase 5 — Normalization

After the emergency vehicle passes the intersection, the system restores the standard adaptive signal timing process.

Control Logic Representation

Input:

- Detected vehicle data
- Vehicle classification results

Processing:

- Check vehicle type
- Compare with emergency vehicle list
- Verify detection confidence

Output:

- Emergency signal priority activation
- Updated traffic signal status

4.2.4 Simulation Using Python (PYGAME)

Simulation using Python with the Pygame library is an important step in the proposed smart traffic management system, as it allows visualization and testing of traffic signal behavior in a controlled virtual environment. The simulation demonstrates how traffic lights change dynamically based on vehicle detection, traffic density, and emergency vehicle priority. This approach helps validate system performance before real-world deployment.

In the proposed system, the Pygame library is used to create a graphical interface representing a traffic intersection with multiple lanes and traffic signals. The simulation displays vehicles moving through the intersection and shows signal changes in real time according to the adaptive signal timing logic. The system receives input data such as vehicle counts and traffic density values and updates the signal states accordingly.

The simulation environment enables testing of different traffic scenarios, including normal traffic flow, high congestion, and emergency vehicle arrival. By visualizing system behavior, the simulation helps evaluate the efficiency of the proposed model, identify potential issues, and improve system reliability. This makes Pygame a useful tool for demonstrating the working of the intelligent traffic control system.

Simulation Using Python (Pygame) – Functional Components

Input

- Traffic density values
- Vehicle count data
- Emergency vehicle detection signal

Intelligent Adaptive Traffic Signal Control Using Computer Vision with Emergency Vehicle Priority

- Signal timing parameters

Processing

- Initialize simulation environment
- Create traffic intersection layout
- Display vehicles and traffic signals
- Apply adaptive signal timing logic
- Update signal status dynamically

Output

- Visual simulation of traffic signals
- Real-time signal changes
- System performance demonstration

Algorithm: Simulation Using Python (Pygame)

Objective

To simulate the behavior of the smart traffic management system and visualize adaptive traffic signal control.

BEGIN

1. Import required Python libraries (Pygame, time, random).
2. Initialize the Pygame simulation environment.
3. Create the traffic intersection layout.
4. Initialize traffic signals for each lane.
5. Generate vehicles in different lanes.
6. Monitor vehicle movement and traffic density.
7. Apply adaptive signal timing logic.
8. Update traffic signal colors (Red, Yellow, Green).
9. Display vehicle movement and signal changes on the screen.
10. Repeat the simulation continuously until the program is stopped.

END

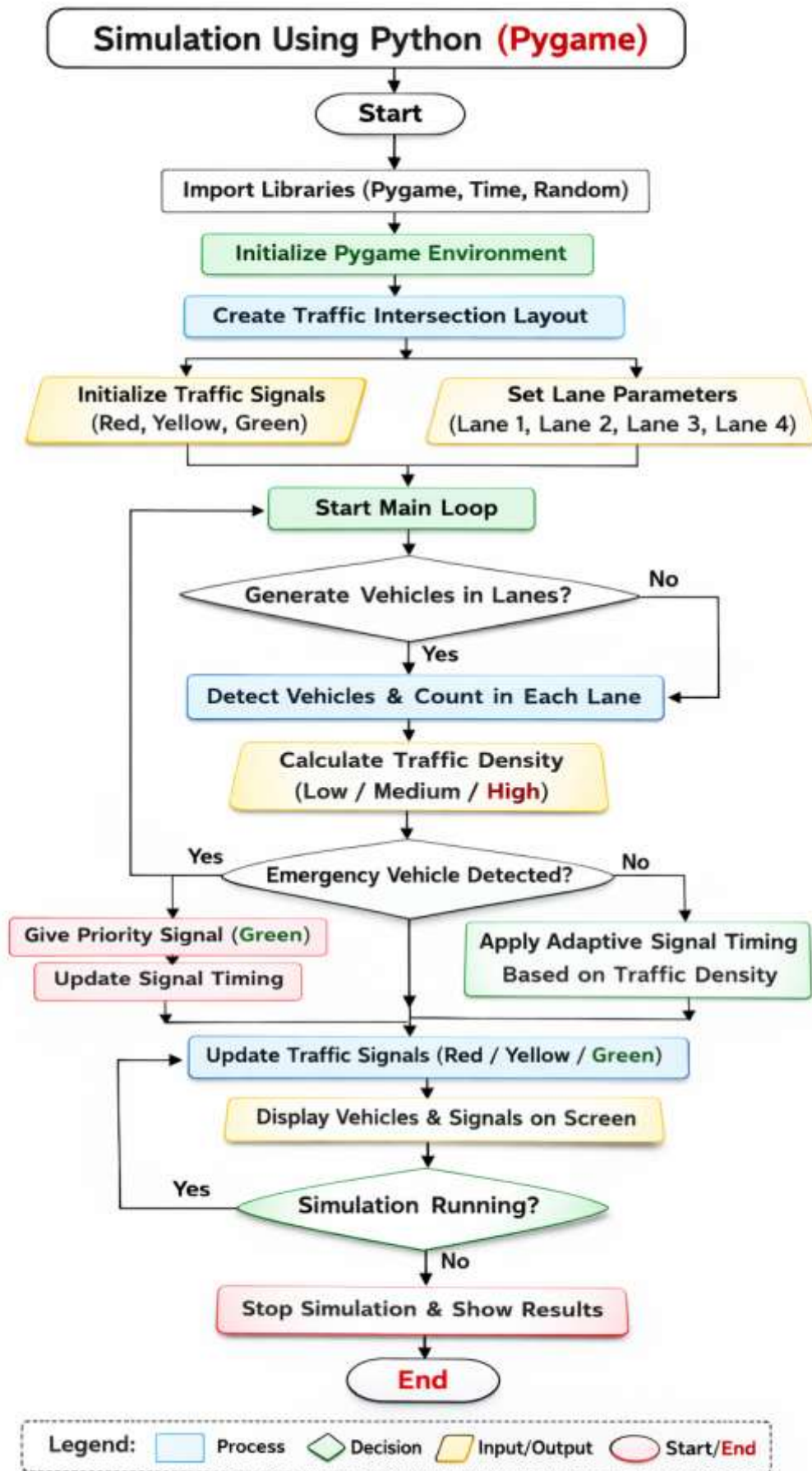


Fig 4.2.4 Simulation Using Python (PYGAME)

4.2.5 System Setup

Software and Hardware Requirements

A. Hardware Requirements

The following are the requirements of hardware

System	: 64-bit Operating System
Storage	: 256 GB SSD or 500 GB HDD
Processor	: Intel® Core™ i5 / i7 or higher
CPU Speed	: 2.4 GHz or above
GPU	: Integrated GPU / NVIDIA GPU (optional for faster processing)
RAM	: Minimum 8 GB (16 GB recommended)
Camera	: HD Camera / CCTV Camera (for video input)
Display	: Monitor with minimum 1366 × 768 resolution

B. Software Requirements

The following are the requirements of software

Operating System	: Windows 10 / Windows 11 / Linux
IDE / Development Tool	: Visual Studio Code / Jupyter Notebook / Google Colab
Coding Language	: Python 3.8 or above
Libraries	: NumPy, OpenCV, Pandas, Matplotlib, Pygame, Ultralytics YOLO
Framework / Tools	: Flask (for web interface, if used)
Dataset / Input	: Traffic video dataset or live camera feed

Installing Python and Visual Studio Code

Step 1: Install Python

1. To download and install Python, visit the official website of Python:
<https://www.python.org/downloads/> and choose the appropriate version for your system.
2. Launch the Python installation file (.exe) once the download is complete.
3. Select "Add Python to PATH" and click Install Now to continue.
4. You can see Python installing at this point.
5. When the installation finishes, a screen will appear showing Setup was successful. Click Close.

Step 2: Install Visual Studio Code

1. Visit the official website of Visual Studio Code:

<https://code.visualstudio.com/>

2. Download the installer and run the setup file.
3. Follow the installation instructions and click Next until the installation is complete.
4. Launch Visual Studio Code after installation.

Algorithm Implementation (Smart Traffic Control System in VS Code)

Step 1: Create a Python Project

Open Visual Studio Code and create a new project folder named:

`Adaptive_Traffic_Signal_Timer`

Step 2: Install Required Libraries

Install the necessary Python libraries using the terminal:

```
pip install opencv-python
```

```
pip install numpy
```

```
pip install matplotlib
```

```
pip install pygame
```

```
pip install ultralytics
```

Step 3: Open the project folder using Visual Studio Code.

Command:

```
cd Adaptive_Traffic_Signal_Timer
```

This command navigates to the project directory where all source files are stored.

Step 4: Load the YOLO Model

Load the pre-trained YOLO model for vehicle detection.

- The model detects vehicles such as:
 - Car
 - Bus
 - Truck
 - Bike
 - Ambulance

Step 5 Provide Video Input Source

The system accepts multiple input sources:

Video File:

`traffic_video.mp4`

Example:

```
cv2.VideoCapture("traffic_video.mp4")
```

Step 6: Perform Video Frame Processing

Each video frame is:

- Captured
- Resized
- Converted into a suitable format
- Prepared for object detection

Step 7: Detect Vehicles Using YOLO

The YOLO model processes each frame and:

- Identifies vehicles
- Draws bounding boxes
- Assigns class labels

Step 8: Perform Lane-wise Vehicle Counting

Vehicles are counted separately in each lane.

The system:

- Assigns vehicles to lanes
- Updates vehicle count
- Stores lane-wise data

Output Example:

- Lane 1: 12 vehicles
- Lane 2: 8 vehicles
- Lane 3: 5 vehicles
- Lane 4: 10 vehicles

Step 9: Estimate Traffic Density

Traffic density is calculated using:

$\text{Traffic Density} = \text{Number of Vehicles} / \text{Lane Capacity}$

The system classifies traffic as:

- Low
- Medium
- High

Step 10: Apply Adaptive Signal Timing

Traffic signal timing is automatically adjusted based on:

- Vehicle count
- Traffic density
- Emergency vehicle detection

Example:

If density is high

 Increase green signal time

Else

 Maintain normal timing

Step 11: Detect Emergency Vehicles

The system detects emergency vehicles such as:

- Ambulance

If detected:

Give signal priority

Switch signal to GREEN

Allow emergency passage

Step 12: Run Traffic Simulation Using Pygame

Run:

`python simulation.py`

The system simulates:

- Traffic intersection
- Moving vehicles
- Traffic signals
- Emergency vehicle priority

Step 13: Run Vehicle Detection Module

Run:

`python vehicle_detection.py`

This will start:

- Video processing
- Vehicle detection
- Lane counting
- Traffic density calculation

Step 14: Final Execution Commands

Run Full System

```
python vehicle_detection.py
```

Run Only Simulation

```
python simulation.py
```

Step 15: Stop the System

The system continues processing until:

- Video ends

OR

- User presses the stop key

Code Snippet

Setup.py

```
from setuptools import setup, find_packages
from setuptools.extension import Extension
from Cython.Build import cythonize
import numpy
import os
import imp
VERSION = imp.load_source('version', os.path.join('.', 'darkflow', 'version.py'))
VERSION = VERSION.__version__
if os.name == 'nt' :
    ext_modules=[
        Extension("darkflow.cython_utils.nms",
            sources=["darkflow/cython_utils/nms.pyx"],
            #libraries=["m"] # Unix-like specific
            include_dirs=[numpy.get_include()]
        ),
        Extension("darkflow.cython_utils.cy_yolo2_findboxes",
            sources=["darkflow/cython_utils/cy_yolo2_findboxes.pyx"],
            #libraries=["m"] # Unix-like specific
            include_dirs=[numpy.get_include()]
        ),
        Extension("darkflow.cython_utils.cy_yolo_findboxes",
            sources=["darkflow/cython_utils/cy_yolo_findboxes.pyx"],
```

```
#libraries=["m"] # Unix-like specific
include_dirs=[numpy.get_include()]
)
]
elif os.name == 'posix' :
    ext_modules=[
        Extension("darkflow.cython_utils.nms",
            sources=["darkflow/cython_utils/nms.pyx"],
            libraries=["m"], # Unix-like specific
            include_dirs=[numpy.get_include()]
        ),
        Extension("darkflow.cython_utils.cy_yolo2_findboxes",
            sources=["darkflow/cython_utils/cy_yolo2_findboxes.pyx"],
            libraries=["m"], # Unix-like specific
            include_dirs=[numpy.get_include()]
        ),
        Extension("darkflow.cython_utils.cy_yolo_findboxes",
            sources=["darkflow/cython_utils/cy_yolo_findboxes.pyx"],
            libraries=["m"], # Unix-like specific
            include_dirs=[numpy.get_include()]
        )
    ]
else :
    ext_modules=[
        Extension("darkflow.cython_utils.nms",
            sources=["darkflow/cython_utils/nms.pyx"],
            libraries=["m"] # Unix-like specific
        ),
        Extension("darkflow.cython_utils.cy_yolo2_findboxes",
            sources=["darkflow/cython_utils/cy_yolo2_findboxes.pyx"],
            libraries=["m"] # Unix-like specific
        ),
        Extension("darkflow.cython_utils.cy_yolo_findboxes",
            sources=["darkflow/cython_utils/cy_yolo_findboxes.pyx"],
```

```
        libraries=["m"] # Unix-like specific
    )
]
setup(
    version=VERSION,
    name='darkflow',
    description='Darkflow',
    license='GPLv3',
    url='https://github.com/thtrieu/darkflow',
    packages = find_packages(),
    scripts = ['flow'],
    ext_modules = cythonize(ext_modules)
)
```

2. Simulation.py

START

INITIALIZE PARAMETERS

```
defaultRed ← 150
defaultYellow ← 5
defaultGreen ← 20
defaultMinimum ← 10
defaultMaximum ← 60
```

```
number_of_signals ← 4
simulationTime ← 300
timeElapsed ← 0
```

```
currentGreen ← 0
nextGreen ← (currentGreen + 1) mod number_of_signals
currentYellow ← 0
```

CREATE list SIGNALS

CREATE dictionary VEHICLES {right, left, up, down}

CREATE queue NORMAL_SEQUENCE ← [0,1,2,3]

CREATE queue EMERGENCY_QUEUE

CREATE set RECENTLY_SERVED

DEFINE vehicle speeds:

car, bus, truck, bike, rickshaw, ambulance

FUNCTION INITIALIZE_SIGNALS

CREATE TS1(red=0, yellow=defaultYellow, green=defaultGreen)

ADD TS1 to SIGNALS

CREATE TS2(red=TS1.red+TS1.yellow+TS1.green,
yellow=defaultYellow,
green=defaultGreen)

ADD TS2 to SIGNALS

FOR i ← 3 to 4

CREATE TSi(red=defaultRed,
yellow=defaultYellow,
green=defaultGreen)

ADD TSi to SIGNALS

END FOR

CALL REPEAT

END FUNCTION

FUNCTION HAS_AMBULANCE_WAITING(direction)

FOR each lane IN direction

FOR each vehicle IN lane

IF vehicle.type = ambulance

AND vehicle NOT crossed_stop_line

RETURN TRUE

END FOR

END FOR

RETURN FALSE

END FUNCTION

FUNCTION CHECK_AND_QUEUE_EMERGENCIES

CREATE CLEANED_QUEUE

FOR each lane IN EMERGENCY_QUEUE

IF ambulance still waiting

ADD lane to CLEANED_QUEUE

END FOR

EMERGENCY_QUEUE ← CLEANED_QUEUE

FOR i ← 0 to number_of_signals-1

IF ambulance detected in lane i

AND i ≠ currentGreen

AND i NOT in queue

```
        ADD i to EMERGENCY_QUEUE
    END FOR
END FUNCTION
```

```
FUNCTION CALCULATE_GREEN_TIME
```

```
    cars ← 0, buses ← 0, trucks ← 0
```

```
    bikes ← 0, rickshaws ← 0
```

```
    FOR each vehicle IN next_lane
```

```
        IF vehicle NOT crossed
```

```
            INCREMENT respective counter
```

```
    END FOR
```

```
    greenTime ←
```

```
        (cars×carTime +
```

```
        buses×busTime +
```

```
        trucks×truckTime +
```

```
        bikes×bikeTime +
```

```
        rickshaws×rickshawTime)
```

```
    ÷ number_of_lanes
```

```
    IF greenTime < defaultMinimum
```

```
        greenTime ← defaultMinimum
```

```
    IF greenTime > defaultMaximum
```

```
        greenTime ← defaultMaximum
```

```
    ASSIGN greenTime to next signal
```

```
END FUNCTION
```

```
FUNCTION UPDATE_SIGNAL_TIMERS
```

```
    FOR each signal i
```

```
        IF i = currentGreen
```

```
            IF currentYellow = OFF
```

```
                DECREASE green by 1
```

```
            ELSE
```

```
                DECREASE yellow by 1
```

```
        ELSE IF i = nextGreen
```

```
            DECREASE red by 1
```

```
    END FOR
```

```
END FUNCTION
```

```
FUNCTION SELECT_NEXT_LANE
```

```
    IF EMERGENCY_QUEUE NOT empty
```

```
    next_lane ← REMOVE first element
ELSE
    next_lane ←
        (currentGreen + 1) mod number_of_signals

RETURN next_lane
END FUNCTION

FUNCTION REPEAT
    WHILE simulation running
        PRINT signal status

        ----- GREEN PHASE -----
        WHILE green_time > 0
            CALL UPDATE_SIGNAL_TIMERS
            CALL CHECK_AND_QUEUE_EMERGENCIES

            IF red_time(next_signal) = detectionTime
                CALL CALCULATE_GREEN_TIME

            WAIT 1 second
        END WHILE

        ----- YELLOW PHASE -----
        currentYellow ← 1

        WHILE yellow_time > 0
            CALL UPDATE_SIGNAL_TIMERS
            WAIT 1 second
        END WHILE

        currentYellow ← 0

        RESET current signal:
            green ← defaultGreen
            yellow ← defaultYellow
            red ← defaultRed

        ADD current_lane to RECENTLY_SERVED

        next_lane ← SELECT_NEXT_LANE

        currentGreen ← next_lane
        nextGreen ←
            (currentGreen + 1) mod number_of_signals
```

```
END WHILE
END FUNCTION

FUNCTION GENERATE_VEHICLES
  WHILE simulation running

    SELECT random vehicle_type
    IF random_probability < 0.01
      vehicle_type ← ambulance
    ELSE
      vehicle_type ←
        car | bus | truck | bike | rickshaw

    SELECT random lane
    SELECT random direction:
      right | down | left | up

    DETERMINE turning behavior
    CREATE vehicle object

    WAIT 0.75 seconds
  END WHILE
END FUNCTION

FUNCTION SIMULATION_TIMER
  WHILE simulation running

    timeElapsed ← timeElapsed + 1
    WAIT 1 second

    IF timeElapsed = simulationTime
      COUNT vehicles passed
      totalVehicles ← sum of all lanes

      vehicles_per_second ←
        totalVehicles ÷ timeElapsed

    STOP program
  END WHILE
END FUNCTION

MAIN PROGRAM
  START THREAD → SIMULATION_TIMER
  START THREAD → INITIALIZE_SIGNALS
```

```
START THREAD → GENERATE_VEHICLES
```

```
WHILE program running
```

```
    DISPLAY background
```

```
    DISPLAY signals (GREEN/YELLOW/RED)
```

```
    DISPLAY vehicle counts
```

```
    MOVE vehicles
```

```
    UPDATE screen
```

```
END WHILE
```

```
END
```

3.vehicle_detection.py

```
import cv2
```

```
from darkflow.net.build import TFNet
```

```
import matplotlib.pyplot as plt
```

```
import os
```

```
options={
```

```
    'model':'./cfg/yolo.cfg',      #specifying the path of model
```

```
    'load':'./bin/yolov2.weights', #weights
```

```
    'threshold':0.3                #minimum confidence factor to create a box, greater than 0.3
```

```
good
```

```
}
```

```
tfnet=TFNet(options)
```

```
inputPath = os.getcwd() + "/test_images/"
```

```
outputPath = os.getcwd() + "/output_images/"
```

```
def detectVehicles(filename):
```

```
    global tfnet, inputPath, outputPath
```

```
    img=cv2.imread(inputPath+filename,cv2.IMREAD_COLOR)
```

```
    # img=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
```

```
    result=tfnet.return_predict(img)
```

```
    # print(result)
```

```
    for vehicle in result:
```

```
        label=vehicle['label'] #extracting label
```

```
        if(label=="car" or label=="bus" or label=="bike" or label=="truck" or
```

```
label=="rickshaw"): # drawing box and writing label
```

```
            top_left=(vehicle['topleft']['x'],vehicle['topleft']['y'])
```

```
            bottom_right=(vehicle['bottomright']['x'],vehicle['bottomright']['y'])
```

```
            img=cv2.rectangle(img,top_left,bottom_right,(0,255,0),3) #green box of width 5
```

Intelligent Adaptive Traffic Signal Control Using Computer Vision with Emergency Vehicle Priority

```
img=cv2.putText(img,label,top_left,cv2.FONT_HERSHEY_COMPLEX,0.5,(0,0,0),1)
#image, label, position, font, font scale, colour: black, line width
outputFilename = outputPath + "output_" +filename
cv2.imwrite(outputFilename,img)
print('Output image stored at:', outputFilename)
# plt.imshow(img)
# plt.show()
# return result
for filename in os.listdir(inputPath):
    if(filename.endswith(".png") or filename.endswith(".jpg") or filename.endswith(".jpeg")):
        detectVehicles(filename)
print("Done!")
```

5.RESULTS AND DISCUSSIONS

5.1 Experimental Results

The Smart Traffic Control System was successfully implemented using Python, computer vision techniques, and a simulation environment. The system processes real-time video input, detects vehicles using a YOLO-based object detection model, performs lane-wise vehicle counting, estimates traffic density, and dynamically controls traffic signals based on traffic conditions.

The simulation results demonstrate that the proposed system efficiently manages traffic flow by adjusting signal timing according to vehicle density. The system also identifies emergency vehicles such as ambulances and provides priority passage by immediately switching the signal to green. The integration of video processing, vehicle detection, and adaptive signal control improves traffic efficiency and reduces congestion at intersections.



Fig 1: Initial Traffic Flow

The experimental results indicate that the Smart Traffic Control System effectively improves traffic management by using real-time vehicle detection and adaptive signal control. The system demonstrates reliable vehicle detection accuracy and consistent lane-wise counting performance under different traffic scenarios. The integration of emergency vehicle detection enhances the safety and responsiveness of the traffic system. The simulation environment validates the functionality of the system logic before real-world deployment. Overall, the proposed system

provides an efficient, scalable, and intelligent solution for modern traffic management and smart city applications.



Fig 2: Increasing Traffic Density

In conclusion, the experimental results demonstrate that the proposed Smart Traffic Control System effectively improves traffic management through real-time vehicle detection, lane-wise counting, and adaptive signal control mechanisms. The system successfully responds to changing traffic conditions by dynamically adjusting signal timing and prioritizing emergency vehicles when necessary. The integration of computer vision techniques and simulation tools ensures accurate monitoring and efficient decision-making at traffic intersections.

Overall, the results confirm that the developed system is reliable, efficient, and capable of reducing traffic congestion while enhancing road safety. The performance observed during testing indicates that the proposed approach can be implemented in real-world traffic management systems and can contribute significantly to the development of intelligent transportation and smart city infrastructure.

5.2 OUTPUT





Fig 5.2.1 - Traffic Signal Control System Simulation

6.Conclusion

The

Smart Traffic Control System developed in this project provides an efficient and intelligent solution for managing traffic at road intersections using computer vision and adaptive signal control techniques. The system integrates video input processing, vehicle detection, lane-wise vehicle counting, and traffic density estimation to monitor traffic conditions in real time. By replacing traditional fixed-time traffic signals with dynamic signal control, the proposed system improves traffic flow and reduces congestion at busy intersections. The implementation of the vehicle detection model enabled accurate identification of vehicles from video streams, while the lane-wise counting and density estimation modules ensured proper analysis of traffic patterns. The adaptive signal timing logic automatically adjusted signal duration based on traffic density, allowing smoother vehicle movement and minimizing unnecessary waiting time. In addition, the emergency vehicle detection feature enhanced the system by providing immediate signal priority, ensuring faster and safer passage for critical vehicles. Overall, the results obtained from testing and simulation confirm that the proposed system is reliable, scalable, and suitable for intelligent transportation and smart city applications. The system demonstrates stable performance, efficient processing, and effective traffic management capabilities. With further improvements such as integration of advanced sensors, cloud connectivity, and real-time monitoring systems, the proposed solution can be extended for large-scale deployment in modern urban traffic management environments.

Future Work

The proposed Smart Traffic Control System can be further enhanced by integrating advanced technologies such as Internet of Things (IoT) devices and smart sensors to improve data collection and communication between traffic signals and control centers. Real-time connectivity between multiple intersections can enable centralized traffic monitoring and coordinated signal control across larger road networks. This integration would allow the system to respond more effectively to changing traffic conditions and improve overall transportation efficiency in urban environments.

Another area for future development is the use of more advanced machine learning and deep learning models to improve vehicle detection accuracy under challenging conditions such as low lighting, heavy rain, fog, or nighttime traffic. The system can also be expanded to recognize additional vehicle categories, pedestrian movement, and traffic violations. Incorporating

predictive analytics techniques can help forecast traffic patterns based on historical data, enabling proactive traffic management and reducing congestion before it occurs.

In addition, the system can be extended to support real-world deployment through cloud-based platforms and mobile applications for remote monitoring and control of traffic signals. Features such as automatic incident detection, emergency response coordination, and integration with public transportation systems can further enhance the functionality of the system. These improvements will make the proposed solution more scalable, reliable, and suitable for implementation in smart city infrastructure and modern intelligent transportation systems.

REFERENCES

- [1] Intelligent Transportation Systems
Boppuru, P. R., Kukatlapalli, P. K., Chowdary, C. R., & Hussain, J.
AI-Based YOLO V4 Intelligent Traffic Light Control System.
Journal of Automation, Mobile Robotics and Intelligent Systems, 2023.
- [2] Sharma, M., Bansal, A., Kashyap, V., & Goyal, P.
Intelligent Traffic Light Control System Based on Traffic Environment Using Deep Learning.
IOP Conference Series: Materials Science and Engineering, 2021.
- [3] Bakirci, M.
Real-Time Vehicle Detection Using YOLOv8-Nano for Intelligent Transportation Systems.
IIETA Journal of Traffic and Safety, 2024.
- [4] Talaat, F. M., Elbalka, R. M., Sweidan, S., & Gamel, S. A.
Smart Traffic Management System Using YOLOv11 for Real-Time Vehicle Detection and Dynamic Flow Optimization in Smart Cities.
Neural Computing and Applications, 2025.
- [5] Kosasi, T., Sihombing, Z. A. L., & Husein, A. M.
YOLO-Based Vehicle Detection: Literature Review.
Journal of Computer Networks, Architecture and High Performance Computing, 2024.
- [6] Sakhare, N., Hedau, M., Vishwakarma, G., Malpure, O., Shah, T., & Ingle, A.
Smart Traffic: Integrating Machine Learning and YOLO for Adaptive Traffic Management System.
International Journal of Intelligent Systems and Applications in Engineering, 2023.
- [7] Wang, Z., Zhan, J., Duan, C., Guan, X., Lu, P., & Yang, K.
A Review of Vehicle Detection Techniques for Intelligent Vehicles.
IEEE Transactions on Neural Networks and Learning Systems, 2022.
- [8] Rahman, R., Azad, Z. B., & Hasan, M. B.
Densely-Populated Traffic Detection Using YOLOv5 and Non-Maximum Suppression Ensembling.
International Conference on Computer Vision Applications, 2021.
- [9] Rahman, Z., Mazumder, A., & Ullah, M. A.
A Real-Time Wrong-Way Vehicle Detection Based on YOLO and Centroid Tracking.
International Journal of Computer Vision Systems, 2022.
- [10] Liang, S., & Wu, H.
Edge YOLO: Real-Time Intelligent Object Detection System Based on Edge-Cloud

Cooperation in Autonomous Vehicles.

IEEE Intelligent Systems, 2022.

[11] Alif, M. A. R.

YOLOv11 for Vehicle Detection: Advancements, Performance, and Applications in Intelligent Transportation Systems.

International Journal of Artificial Intelligence Research, 2024.

[12] Chen, C., Wang, C., Liu, B., He, C., Cong, L., & Wan, S.

Edge Intelligence Empowered Vehicle Detection and Image Segmentation for Autonomous Vehicles.

IEEE Transactions on Intelligent Transportation Systems, 2023.

[13] Yuan, Y., Cheng, H., & Sester, M.

Keypoints-Based Deep Feature Fusion for Cooperative Vehicle Detection of Autonomous Driving.

IEEE Robotics and Automation Letters, 2022.

[14] Chen, X., et al.

Vehicle Detection from Road Image Sequences for Intelligent Traffic Scheduling.

Computers & Electrical Engineering, 2021.

[15] Digital Signal Processing Research Group.

Utilizing YOLOv8 for Enhanced Traffic Monitoring in Intelligent Transportation Systems Applications.

Digital Signal Processing Journal, 2024.